



Département des Technologies de l'information et de la
communication (TIC)
Filière Informatique et systèmes de communication
Orientation Sécurité informatique

Rapport de recherche

Caméras virtuelles et diffusion de flux vidéo

Étudiant

Dylan Oliveira Ramos

Enseignant responsable

Prof. Jean-Marc Bost

Année académique

2025-2026

Yverdon-les-Bains, le 08.06.2026

Table des matières

1	Introduction	4
2	Comparaison des solutions	4
3	Caméras virtuelles sous Linux	4
3.1	Installation	4
3.2	Création d'une caméra virtuelle	4
3.3	Diffusion d'un flux vidéo vers la caméra virtuelle	5
3.3.1	Rechargement du module	5
3.4	Script d'automatisation	5
4	Caméras virtuelles sous Windows	6
4.1	Installation	6
4.2	Diffusion d'un flux vidéo vers la caméra virtuelle	6
4.3	Automatisation	7
4.3.1	Création de la source vidéo	7
4.3.2	Diffusion d'un flux vidéo vers la caméra virtuelle	7
5	Utilisation d'une caméra virtuelle sur un émulateur Android (Linux)	8
5.1	Installation de Genymotion	8
5.2	Utilisation de la caméra virtuelle de la machine hôte	9
6	Librairies Python pour la diffusion de flux vidéo vers une caméra virtuelle	9
6.1	pyvirtualcam	10
6.2	ffmpeg-python	12
6.3	Comparaison des librairies	12

Table des figures

Tableau 1	Comparaison des différentes solutions de caméras virtuelles.	4
Liste 1	Installation de <code>v4l2loopback</code> et <code>FFmpeg</code> sur Ubuntu.	4
Liste 2	Création d'une caméra virtuelle avec <code>v4l2loopback</code>	5
Liste 3	Énumération des caméras disponibles avec <code>v4l2loopback</code>	5
Liste 4	Diffusion d'une vidéo sur une caméra virtuelle sous Linux.	5
Liste 5	Rechargement du module <code>v4l2loopback</code>	5
Liste 6	Script d'automatisation pour diffuser une vidéo sur une caméra virtuelle sous Linux.	6
Liste 7	Lancement du script d'automatisation pour diffuser une vidéo sur la caméra virtuelle.	6
Liste 8	Énumération des périphériques vidéo disponibles avec <code>FFmpeg</code> sur Windows.	7
Liste 9	Diffusion d'une vidéo sur une caméra virtuelle sous Windows en utilisant <code>FFmpeg</code>	7
Fig. 1	Interface de <code>Genymotion</code> après la création d'un appareil virtuel.	8
Fig. 2	Utilisation de la caméra virtuelle de la machine hôte dans <code>Genymotion</code>	9
Liste 10	Création d'un environnement virtuel Python et installation des dépendances. . .	10
Liste 11	Exemple de code Python utilisant <code>pyvirtualcam</code> pour diffuser une vidéo sur une caméra virtuelle.	11
Liste 12	Lancement du script Python pour diffuser une vidéo sur la caméra virtuelle. . .	11
Liste 13	Création d'un environnement virtuel Python et installation de <code>ffmpeg-python</code> . .	12
Liste 14	Exemple de code Python utilisant <code>ffmpeg-python</code> pour diffuser une vidéo sur une caméra virtuelle.	12
Liste 15	Lancement du script Python pour diffuser une vidéo sur la caméra virtuelle. . .	12
Fig. 3	Comparaison de <code>pyvirtualcam</code> et <code>ffmpeg-python</code> pour diffuser un flux vidéo vers une caméra virtuelle.	13

1 Introduction

Pour pouvoir tromper les sites de vérification d'identité, il faut trouver un moyen de diffuser la vidéo générée vers une caméra détectée comme réelle par ceux-ci. La solution la plus simple est d'utiliser une caméra virtuelle, qui est un périphérique logiciel simulant une caméra physique.

Chaque OS a sa propre manière de gérer les caméras virtuelles. Sous Linux, il faut passer par un module du noyau dédié, alors que sous Windows, il faut développer son propre pilote de caméra virtuelle.

Ce document présente les différentes solutions existantes pour créer des caméras virtuelles sur les différents OS, ainsi que les étapes nécessaires pour diffuser un flux vidéo vers celles-ci.

2 Comparaison des solutions

L'objectif n'est pas de développer une caméra virtuelle de zéro, mais plutôt d'utiliser des solutions existantes. Le tableau ci-dessous compare les différentes solutions permettant de créer des caméras virtuelles :

Solution	OS	Open source	Avantages	Inconvénients
v4l2loopback	Linux	Oui	Natif au noyau Linux, faible latence	Linux uniquement
OBS Studio	Linux, Windows, macOS	Oui	Abstraction de l'OS	Nécessite des actions manuelles
UnityCapture	Windows	Oui	Caméra Windows native	Windows uniquement
CoreMedia IO	macOS	Non	Caméra macOS native	macOS uniquement

Tableau 1. – Comparaison des différentes solutions de caméras virtuelles.

3 Caméras virtuelles sous Linux

Sous Linux, il existe un module noyau appelé **v4l2loopback** permettant de créer des périphériques vidéo virtuels. Avec **FFmpeg**, il est ensuite possible de diffuser un flux vidéo vers ces périphériques, qui seront détectés comme des caméras réelles par les applications.

Les commandes qui vont suivre ont été effectuées sur une machine **Ubuntu 26.04**.

3.1 Installation

```
1 sudo apt install v4l2loopback-dkms v4l2loopback-utils ffmpeg
```

Liste 1. – Installation de **v4l2loopback** et **FFmpeg** sur Ubuntu.

3.2 Création d'une caméra virtuelle

La commande ci-dessous crée une caméra virtuelle appelée **VirtualCam** :

```
1 sudo modprobe v4l2loopback devices=1 video_nr=2 card_label="VirtualCam"
   exclusive_caps=1
```

Liste 2. – Création d’une caméra virtuelle avec `v4l2loopback`.

- `modprobe v4l2loopback` : charge le module noyau.
- `devices=1` : crée 1 périphérique virtuel.
- `video_nr=2` : spécifie le numéro du périphérique (erreur s’il existe déjà).
- `card_label="VirtualCam"` : spécifie le nom du périphérique.
- `exclusive_caps=1` : rend la caméra compatible avec les applications (simule une caméra réelle).

Il est ensuite possible de lister les caméras disponibles :

```
1 v4l2-ctl --list-devices
```

Liste 3. – Énumération des caméras disponibles avec `v4l2loopback`.

3.3 Diffusion d’un flux vidéo vers la caméra virtuelle

La commande ci-dessous joue la vidéo `video.mp4` en boucle sur la caméra virtuelle :

```
1 ffmpeg -re -stream_loop -1 -i video.mp4 -f v4l2 -pix_fmt yuv420p /dev/video2
```

Liste 4. – Diffusion d’une vidéo sur une caméra virtuelle sous Linux.

- `re` : temps réel (simule une vraie caméra sans envoyer toutes les images en une fois).
- `stream_loop -1` : boucle indéfiniment.
- `i video.mp4` : spécifie la vidéo à lancer.
- `f v4l2` : spécifie le format de sortie.
- `pix_fmt yuv420p` : spécifie le format de la vidéo (utilisé par les vraies caméras).
- `/dev/video2` : spécifie le périphérique de sortie.

3.3.1 Rechargement du module

Avant chaque diffusion de vidéo, il faut recharger le module `v4l2loopback` pour éviter les problèmes de conflits. Cela garantit que la caméra virtuelle est correctement réinitialisée et prête à recevoir le flux vidéo :

```
1 sudo modprobe -r v4l2loopback
2 sudo modprobe v4l2loopback devices=1 video_nr=2 card_label="VirtualCam"
   exclusive_caps=1
```

Liste 5. – Rechargement du module `v4l2loopback`.

3.4 Script d’automatisation

Le script ci-dessous automatise le processus de rechargement du module et de diffusion de la vidéo sur la caméra virtuelle :

```

1  #!/bin/bash
2
3  # Check if video file argument is provided
4  if [ $# -eq 0 ]; then
5      echo "Error: Please provide a video file path"
6      echo "Usage: $0 <video_file>"
7      exit 1
8  fi
9
10 VIDEO_FILE="$1"
11
12 # Check if the video file exists
13 if [ ! -f "$VIDEO_FILE" ]; then
14     echo "Error: Video file '$VIDEO_FILE' not found"
15     exit 1
16 fi
17
18 echo "Removing v4l2loopback module..."
19 sudo modprobe -r v4l2loopback
20
21 echo "Loading v4l2loopback module with virtual camera..."
22 sudo modprobe v4l2loopback devices=1 video_nr=2 card_label="VirtualCam"
23     exclusive_caps=1
24
25 # Small delay to ensure the device is ready
26 sleep 1
27
28 echo "Starting video stream to virtual camera..."
29 echo "Press Ctrl+C to stop streaming"
30 ffmpeg -re -stream_loop -1 -i "$VIDEO_FILE" -f v4l2 -pix_fmt yuv420p /dev/video2

```

Liste 6. – Script d’automatisation pour diffuser une vidéo sur une caméra virtuelle sous Linux. Pour l’utiliser, il suffit de lancer la commande suivante :

```

1  ./runvirtcam.sh video.mp4

```

Liste 7. – Lancement du script d’automatisation pour diffuser une vidéo sur la caméra virtuelle.

4 Caméras virtuelles sous Windows

Sous Windows, contrairement à Linux, il n’existe pas d’outils en ligne de commande permettant de créer des caméras virtuelles. Pour pouvoir créer une caméra virtuelle, il faut soit développer un pilote customisé¹, soit utiliser un logiciel proposant cette fonctionnalité. OBS Studio par exemple, utilise la scène comme caméra virtuelle et permet de diffuser un flux vidéo vers celle-ci.

Les instructions qui vont suivre ont été effectuées sur une machine virtuelle **Windows 10 22H2**.

4.1 Installation

Télécharger et installer OBS Studio : <https://obsproject.com/>. Une fois le logiciel installé, la caméra virtuelle est automatiquement créée et est disponible sous le nom de OBS Virtual Camera.

4.2 Diffusion d’un flux vidéo vers la caméra virtuelle

1. Lancer OBS Studio.
2. Dans la section Sources, cliquer sur le bouton + et sélectionner Media Source.

¹<https://medium.com/@sbonnet.dev/how-to-build-a-virtual-camera-under-linux-and-windows-7af0e6433796#3914>

3. Insérer le nom de la source.
4. Cocher **Local File** et **Loop**, sélectionner la vidéo à lancer dans **Local File**, puis cliquer sur **OK**.
5. Dans la section **Controls**, cliquer sur **Start Virtual Camera**.

La vidéo est maintenant diffusée en boucle sur la caméra virtuelle et est accessible aux applications, mais attention, la caméra n'apparaît pas dans le gestionnaire de périphériques Windows.

En effet, cela est dû au fait que c'est une caméra logicielle qui utilise le framework DirectShow, elle est donc enregistrée dynamiquement à l'exécution et n'est pas détectée comme un périphérique physique par l'OS². Il est néanmoins possible de vérifier qu'elle existe vraiment en utilisant FFmpeg pour lister les périphériques vidéo disponibles :

```
1 ffmpeg.exe -list_devices true -f dshow -i dummy
```

Liste 8. – Énumération des périphériques vidéo disponibles avec FFmpeg sur Windows.

4.3 Automatisation

Comme vu dans le point précédent, la création de la source vidéo nécessite des actions manuelles, mais une fois celle-ci créée, il est tout à fait possible d'automatiser le lancement des vidéos via la ligne de commande. Comme pour Linux, il est possible de diffuser un flux vidéo vers la caméra virtuelle en utilisant FFmpeg, cependant, pour que cela fonctionne avec OBS Studio, le flux doit être envoyé via le protocole UDP, ce qui ajoute de la latence.

4.3.1 Création de la source vidéo

1. Lancer OBS Studio.
2. Dans la section **Sources**, cliquer sur le bouton **+** et sélectionner **Media Source**.
3. Insérer le nom de la source.
4. Décocher **Local File**.
5. Dans **Input**, entrer : `udp://127.0.0.1:1234` et cliquer sur **OK**.

Cette source vidéo est maintenant prête à recevoir un flux vidéo via UDP sur le port 1234.

4.3.2 Diffusion d'un flux vidéo vers la caméra virtuelle

La commande ci-dessous joue la vidéo `video.mp4` en boucle sur la caméra virtuelle de OBS Studio :

```
1 ffmpeg.exe -re -stream_loop -1 -i video.mp4 -c:v libx264 -preset ultrafast -tune zerolatency -f mpegts "udp://127.0.0.1:1234?pkt_size=1316"
```

Liste 9. – Diffusion d'une vidéo sur une caméra virtuelle sous Windows en utilisant FFmpeg.

- `re` : temps réel (simule une vraie caméra sans envoyer toutes les images en une fois).
- `stream_loop -1` : boucle indéfiniment.
- `i video.mp4` : spécifie la vidéo à lancer.
- `c:v libx264` : encode la vidéo en H.264 (format compatible avec OBS).
- `preset ultrafast` : utilise le preset d'encodage le plus rapide (réduit la qualité mais permet d'avoir une latence plus faible).
- `tune zerolatency` : optimise l'encodage pour réduire la latence.
- `f mpegts` : spécifie le format de sortie (MPEG-TS est un format de conteneur compatible avec le streaming en direct).

²<https://medium.com/deelvin-machine-learning/how-does-obs-virtual-camera-plugin-work-on-windows-e92ab8986c4e#0878>

- `udp://127.0.0.1:1234?pkt_size=1316` : spécifie l'adresse et le port de destination du flux UDP, ainsi que la taille des paquets (1316 est une taille courante pour le streaming en direct).

5 Utilisation d'une caméra virtuelle sur un émulateur Android (Linux)

Certains sites demandent de vérifier l'identité de l'utilisateur sur un téléphone plutôt que sur un ordinateur, il faut donc que la caméra virtuelle de la machine hôte soit détectée par les émulateurs Android. L'exemple ci-dessous utilise Genymotion comme émulateur Android (sous Linux) car il est plus rapide et plus léger que l'émulateur de Android Studio.

5.1 Installation de Genymotion

1. Télécharger Genymotion : <https://www.genymotion.com/product-desktop/download/>.
2. Installer Genymotion.

```
1 chmod +x genymotion-3.10.0-linux_x64.run
2 ./genymotion-3.10.0-linux_x64.run
```

3. Lancer Genymotion et créer un compte.
4. Dans l'onglet Devices, cliquer sur Create puis sur Install en laissant les paramètres par défaut.

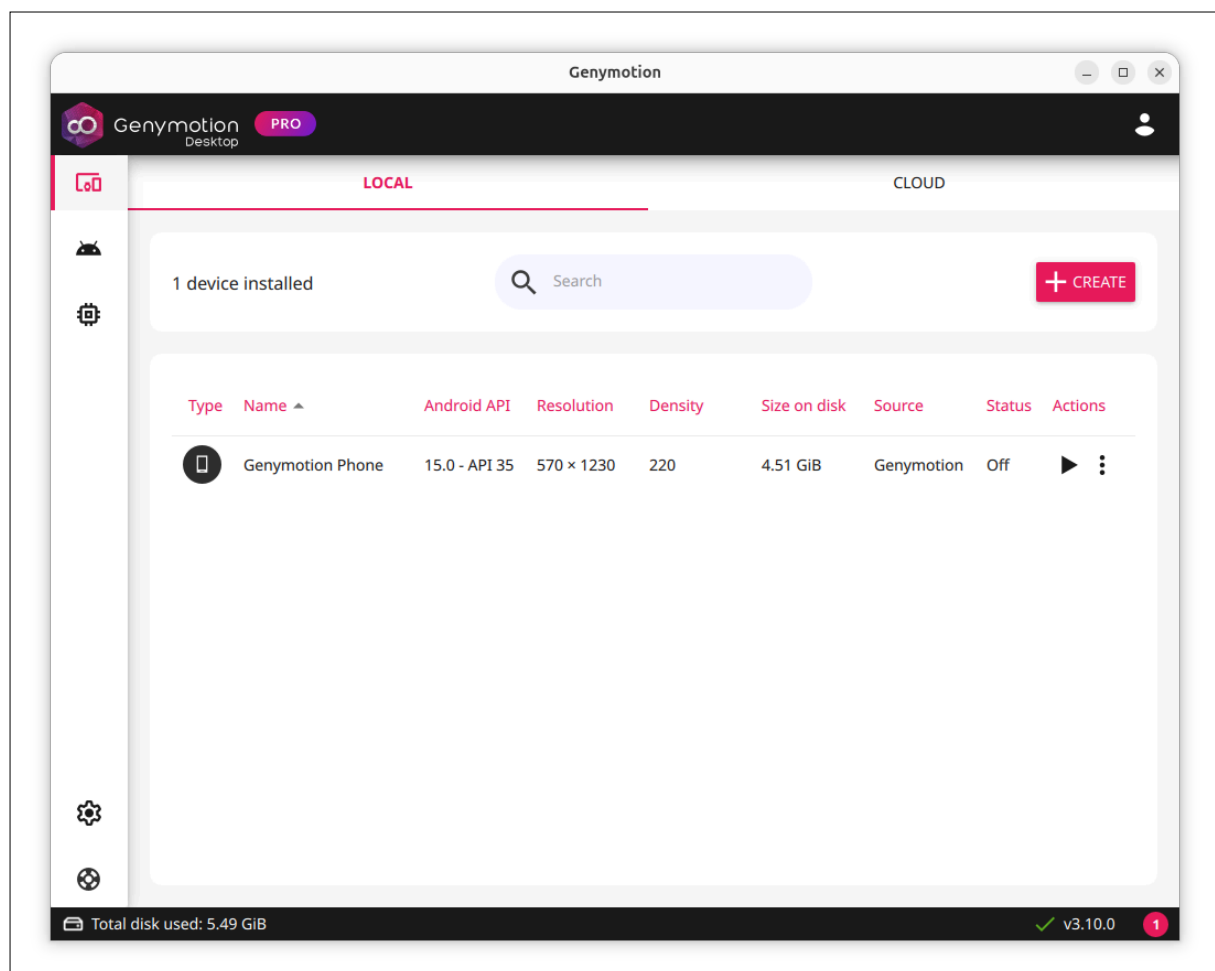


Fig. 1. – Interface de Genymotion après la création d'un appareil virtuel.

5.2 Utilisation de la caméra virtuelle de la machine hôte

1. Diffuser une vidéo sur la caméra virtuelle de la machine hôte (voir [Chapitre 3.3](#))
2. Lancer l'appareil virtuel créé précédemment (bouton Play).
3. Une fois l'appareil démarré, cliquer sur **Media injection** (icône de la caméra dans la barre d'outils à droite).
4. Sélectionner la caméra virtuelle dans la section **Video** de **Inputs mapping**.

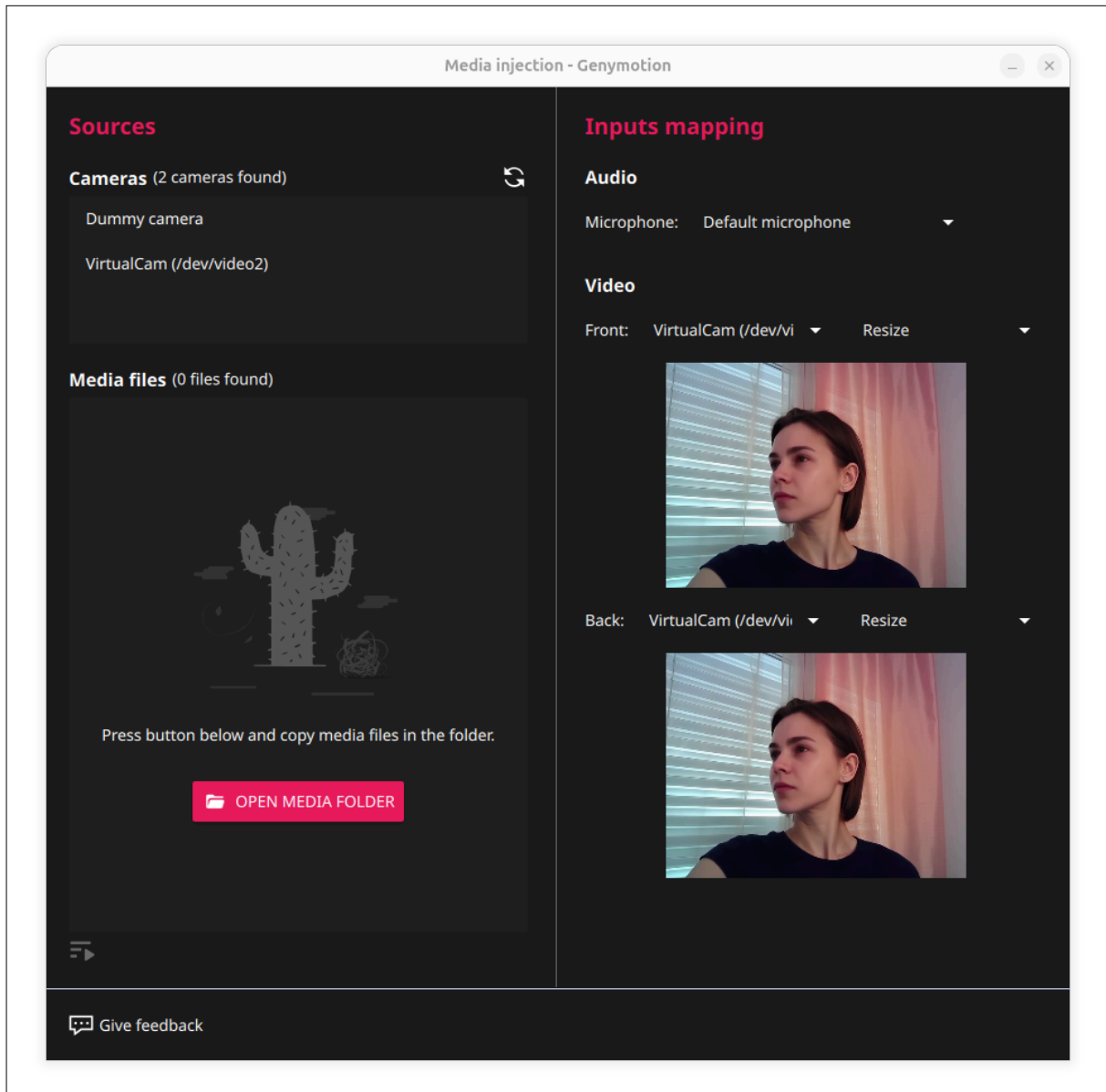


Fig. 2. – Utilisation de la caméra virtuelle de la machine hôte dans Genymotion.

6 Bibliothèques Python pour la diffusion de flux vidéo vers une caméra virtuelle

Pour développer le démonstrateur (CLI), il est nécessaire de pouvoir diffuser un flux vidéo vers une caméra virtuelle de manière programmatique. Il existe plusieurs bibliothèques Python permettant de faire cela, avec chacune leurs avantages et leurs inconvénients.

6.1 pyvirtualcam

La librairie Python `pyvirtualcam` permet d'envoyer un flux vidéo vers une caméra virtuelle existante, que ce soit sur Linux, Windows ou macOS. Elle a le grand avantage de gérer automatiquement les différentes étapes nécessaires pour que le flux vidéo soit correctement redirigé vers la caméra virtuelle. Ainsi, il est possible de s'affranchir de l'utilisation de `FFmpeg` et de la configuration de `OBS Studio`, le tout en étant compatible avec tous les OS.

Mais attention, `pyvirtualcam` nécessite que les caméras virtuelles soient déjà créées, ce qui implique de devoir installer une solution de caméra virtuelle adaptée à son OS (voir [Chapitre 3.2](#) pour Linux et [Chapitre 4.1](#) pour Windows et macOS).

L'exemple ci-dessous montre comment utiliser `pyvirtualcam` sur Windows. Dans cet exemple, la caméra virtuelle de `OBS Studio` est utilisée (voir le [Chapitre 4.1](#) pour l'installation).

Création de l'environnement virtuel :

```
1 python -m venv .venv
2 .venv\Scripts\activate
3 pip install pyvirtualcam opencv-python
```

Liste 10. – Création d'un environnement virtuel Python et installation des dépendances.

Exemple de code :

Source : <https://github.com/letmaik/pyvirtualcam/blob/main/examples/video.py>

```

1 # This script plays back a video file on the virtual camera.
2 # It also shows how to:
3 # - select a specific camera device
4 # - use BGR as pixel format
5
6 import argparse
7 import pyvirtualcam
8 from pyvirtualcam import PixelFormat
9 import cv2
10
11 parser = argparse.ArgumentParser()
12 parser.add_argument("video_path", help="path to input video file")
13 parser.add_argument("--fps", action="store_true", help="output fps every second")
14 parser.add_argument("--device", help="virtual camera device, e.g. /dev/video0 (optional)")
15 args = parser.parse_args()
16
17 video = cv2.VideoCapture(args.video_path)
18 if not video.isOpened():
19     raise ValueError("error opening video")
20 length = int(video.get(cv2.CAP_PROP_FRAME_COUNT))
21 width = int(video.get(cv2.CAP_PROP_FRAME_WIDTH))
22 height = int(video.get(cv2.CAP_PROP_FRAME_HEIGHT))
23 fps = video.get(cv2.CAP_PROP_FPS)
24
25 with pyvirtualcam.Camera(width, height, fps, fmt=PixelFormat.BGR,
26                          device=args.device, print_fps=args.fps) as cam:
27     print(f'Virtual cam started: {cam.device} ({cam.width}x{cam.height} @ {cam.fps}fps)')
28     count = 0
29     while True:
30         # Restart video on last frame.
31         if count == length:
32             count = 0
33             video.set(cv2.CAP_PROP_POS_FRAMES, 0)
34
35         # Read video frame.
36         ret, frame = video.read()
37         if not ret:
38             raise RuntimeError('Error fetching frame')
39
40         # Send to virtual cam.
41         cam.send(frame)
42
43         # Wait until it's time for the next frame
44         cam.sleep_until_next_frame()
45
46         count += 1

```

Liste 11. – Exemple de code Python utilisant pyvirtualcam pour diffuser une vidéo sur une caméra virtuelle.

Utilisation :

```

1 python video.py video.mp4

```

Liste 12. – Lancement du script Python pour diffuser une vidéo sur la caméra virtuelle.

La vidéo video.mp4 est maintenant diffusée en boucle sur la caméra virtuelle de OBS Studio.

6.2 ffmpeg-python

La librairie `ffmpeg-python` est une interface Python qui permet de construire des commandes `FFmpeg` de manière programmatique. L'avantage de cette librairie est qu'elle offre plus de flexibilité que `pyvirtualcam`, notamment en permettant de rogner la vidéo, changer sa résolution, changer le format des pixels, etc. Un autre avantage est qu'elle permet de diffuser une image statique comme une vidéo, ce qui est utile pour simuler une caméra scanant une pièce d'identité. Par contre, elle implique l'utilisation de `FFmpeg` or, comme vu précédemment, l'utilisation de `FFmpeg` avec `OBS Studio` sur Windows implique de devoir diffuser le flux vidéo via `UDP`, ce qui ajoute de la latence.

L'exemple ci-dessous montre comment utiliser `ffmpeg-python` sur Linux. Pour que cet exemple fonctionne, il faut avoir une caméra virtuelle disponible (voir le [Chapitre 3.2](#) pour l'installation) ainsi que `FFmpeg` installé.

Création de l'environnement virtuel :

```
1 python3 -m venv .venv
2 source .venv/bin/activate
3 pip install ffmpeg-python
```

Liste 13. – Création d'un environnement virtuel Python et installation de `ffmpeg-python`.

Exemple de code :

```
1 import ffmpeg
2 import sys
3
4
5 def main(arg1):
6     ffmpeg.input(arg1, re=None, stream_loop=-1).filter("scale", 640,
7     480).filter("fps", 30).output("/dev/video2", format="v4l2",
8     pix_fmt="yuv420p").run()
9
10 if __name__ == "__main__":
11     if len(sys.argv) != 2:
12         print("Usage: python main.py <video_file>")
13         sys.exit(1)
14     main(sys.argv[1])
```

Liste 14. – Exemple de code Python utilisant `ffmpeg-python` pour diffuser une vidéo sur une caméra virtuelle.

Utilisation :

```
1 python main.py video.mp4
```

Liste 15. – Lancement du script Python pour diffuser une vidéo sur la caméra virtuelle.

La vidéo passée en argument est maintenant diffusée en boucle sur la caméra virtuelle `/dev/video2`.

6.3 Comparaison des librairies

Les librairies `pyvirtualcam` et `ffmpeg-python` permettent d'atteindre le même objectif, diffuser une vidéo sur une caméra virtuelle créée préalablement. `pyvirtualcam` est multiplateforme mais moins flexible que `ffmpeg-python` car elle ne permet pas d'éditer les vidéos ni de diffuser des

images statiques. Cependant, `ffmpeg-python` bien que multiplateforme également, est moins efficace sur Windows lorsqu'elle est utilisée avec `OBS Studio` car elle nécessite de diffuser le flux vidéo via UDP, ce qui ajoute de la latence.

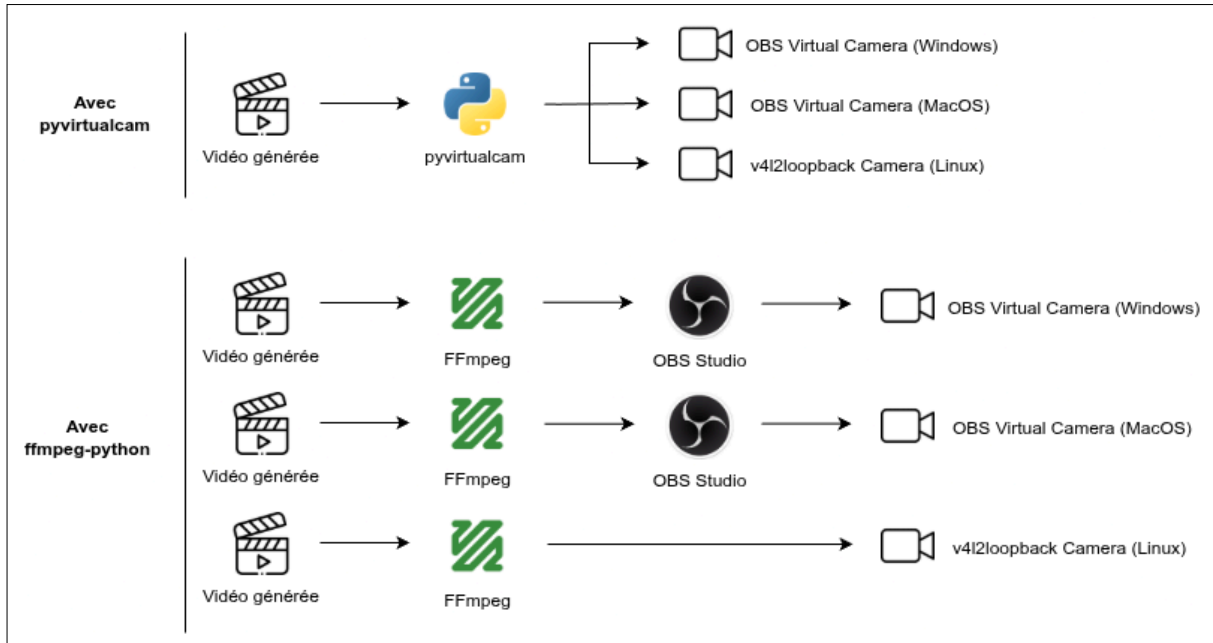


Fig. 3. – Comparaison de `pyvirtualcam` et `ffmpeg-python` pour diffuser un flux vidéo vers une caméra virtuelle.